

Predicción de Incumplimiento de Créditos

Proyecto Integrador

Proyecto de Ciencia de Datos end-to-end desarrollado para el equipo de Datos y Analítica de una entidad financiera. Un Científico de Datos Junior Advanced construye, despliega y monitorea un modelo predictivo que, a partir del historial crediticio, anticipa si un cliente nuevo pagará a tiempo o incurrirá en incumplimiento.

El repositorio cubre el ciclo de MLOps completo: calidad y preparación de datos, EDA, ingeniería de características, entrenamiento y selección de modelo, despliegue como API (FastAPI + Docker), envío de predicciones, monitoreo de data drift y un tablero de visualización en Streamlit.

Tabla de contenido

0. [Ejecución rápida \(paso a paso\)](#)
1. [Descripción general y caso de negocio](#)
2. [Flujo general del proyecto](#)
3. [Arquitectura y estructura de carpetas](#)
4. [Datos de entrada y preparación — Cargar_datos.py](#)
5. [Análisis exploratorio de datos \(EDA\) — Comprehension_eda.py](#)
6. [Ingeniería de características — ft_engineering.py](#)
7. [Entrenamiento y evaluación de modelos — model_training_evaluation.py](#)
8. [Model Deployment — model_deploy.py](#)
9. [API de predicción \(FastAPI\) — API.py](#)
10. [Envío de registros a la API — enviar_predicciones_API.py](#)
11. [Model Monitoring y Data Drift — model_monitoring.py](#)
12. [Aplicación de Streamlit — APP.py](#)
13. [Consideraciones metodológicas](#)
14. [Contenerización con Docker](#)
15. [Versionamiento con Git y GitHub](#)
16. [Stack tecnológico](#)
17. [Conclusión](#)

Ejecución rápida (paso a paso)

Guía de referencia para revisión del proyecto. El detalle de cada etapa está desarrollado en las secciones siguientes.

Regla fundamental: todos los scripts deben permanecer en la carpeta raíz del proyecto (junto con los archivos/carpetas que consumen) y ejecutarse desde esa carpeta. Las rutas se construyen de forma relativa a la ubicación del propio script, por lo que moverlo, renombrar artefactos o ejecutarlo desde otra ubicación produce errores de archivo no encontrado. La carpeta raíz puede vivir en cualquier ruta del computador; lo único que debe conservarse es la estructura interna.

Preparación del entorno

```
python -m venv venv
venv\Scripts\activate      # Windows
source venv/bin/activate   # Linux / macOS

pip install -r requirements.txt
```

Orden de ejecución

#	Comando	Qué hace
1	python Cargar_datos.py	Limpia Base_de_datos.csv → genera Base_de_datos_preparada.csv
2	python Comprension_eda.py	Regenera el EDA completo (reportes, CSV y 10 gráficos)
3	python ft_engineering.py	Genera train/test transformados + pipeline de preprocesamiento
4	python model_training_evaluation.py	Entrena y compara los 3 modelos, guarda modelo_final.joblib
5	python model_deploy.py	Aplica el modelo final y genera predicciones/reportes de validación
6	uvicorn API:app --host 127.0.0.1 -- port 8000	Levanta la API — dejar corriendo en Terminal 1
7	Probar POST /predict/batch en /docs con el JSON de ejemplo (sección 9)	Verificación manual de que el servicio responde correctamente
8	En Terminal 2, sin cerrar la API: python enviar_predicciones_API.py	Envía hasta 100 registros al endpoint batch → genera predicciones_api_monitoring.csv
9	python model_monitoring.py	Valida las predicciones, calcula métricas y data drift
10	python -m streamlit run APP.py	Levanta el tablero de visualización del monitoreo
11 (opcional)	docker build -t modelo-creditos-api . / docker run -d -p 8000:8000 ...	Empaqueta y sirve la API en un contenedor, alternativa al paso 6

Terminal 1

```
uvicorn API:app
--host 127.0.0.1 --port 8000 →
(se mantiene abierta)
```

Terminal 2

```
(esperar a que la API esté arriba)
python enviar_predicciones_API.py
(se ejecuta y termina)
```

La API debe permanecer activa mientras se prueba /predict/batch desde /docs y mientras corre enviar_predicciones_API.py. Puede cerrarse después, ya que model_monitoring.py y APP.py solo leen los archivos que ya quedaron generados.

Descripción general y caso de negocio

Has iniciado tu labor en el equipo de Datos y Analítica de una empresa financiera, desempeñándote como Científico de Datos Junior Advanced. Tu primera asignación consiste en desarrollar un modelo predictivo mediante técnicas de aprendizaje automático, utilizando información histórica de créditos, con el objetivo de anticipar el comportamiento de nuevos usuarios.

La empresa opera bajo un esquema estructurado de proyectos, en el cual cada iniciativa debe seguir una arquitectura de carpetas estrictamente definida. Esta estructura no puede ser modificada, ya que los procesos de despliegue a producción están automatizados a través de pipelines de validación en Jenkins. Cualquier alteración en la organización de carpetas podría generar retrasos significativos en el paso a producción.

La variable objetivo utilizada en el proyecto es Pago_atiempo. La clasificación utilizada por los scripts interpreta la clase 0 como posible incumplimiento y la clase 1 como pago a tiempo. El problema presenta una distribución de clases desbalanceada, por lo que el análisis no debe depender únicamente de Accuracy

Prioridad de negocio: minimizar el riesgo de originar créditos que no se pagarán. El criterio de selección del modelo prioriza el recall de la clase 0 por encima de la exactitud general.

Flujo general del proyecto

Etapas	Script principal	Entrada principal	Resultado principal
Preparación y calidad	Cargar_datos.py	Base_de_datos.csv	Base_de_datos_preparada.csv
EDA	Comprension_eda.py	Base_de_datos_preparada.csv	Reporte EDA, CSV y 10 gráficos
Feature Engineering	ft_engineering.py	Base_de_datos_preparada.csv	Train/test transformados y pipeline
Entrenamiento	model_training_evaluation.py	Train/test transformados	Modelos evaluados y modelo_final.joblib
Deployment	model_deploy.py	Modelo + pipeline + datos	Predicciones y reportes
Servicio	API.py	Modelo + pipeline	Endpoints de predicción
Envío a API	enviar_predicciones_API.py	Base_de_datos.csv	predicciones_api_monitoring.csv
Monitoring	model_monitoring.py	Base preparada + predicciones API	Métricas, drift y análisis temporal
Dashboard	APP.py	Resultados de monitoring	Interfaz Streamlit

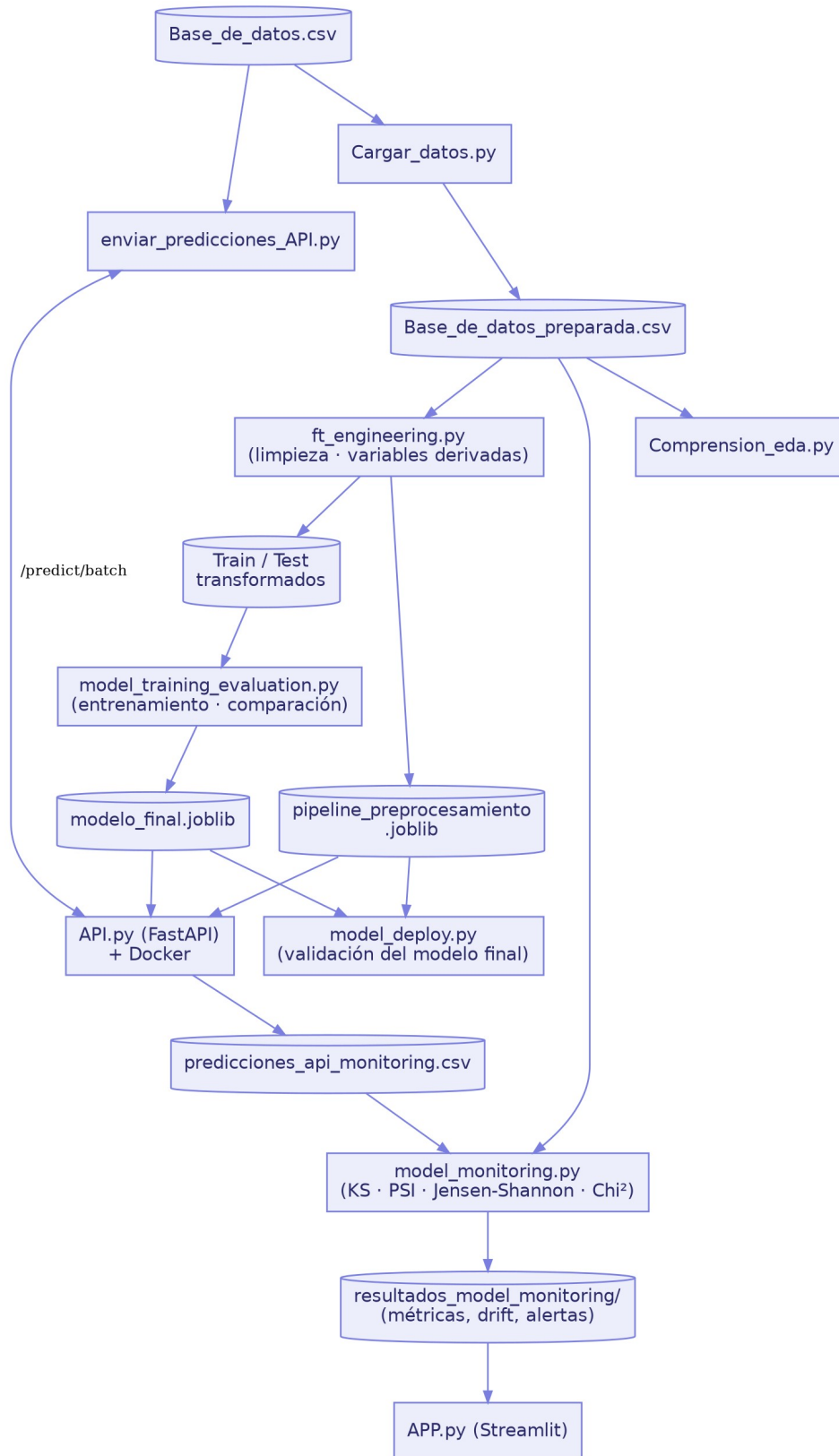
Los scripts construyen sus rutas a partir de su propia ubicación o de rutas relativas: mover un archivo, renombrar un artefacto o ejecutar desde otra carpeta puede provocar errores de archivo no encontrado.

Arquitectura y estructura de carpetas

```
proyecto-credito-ml/
├── Base_de_datos.csv                # Dataset crudo (10.763 filas x 23 columnas)
├── Base_de_datos_preparada.csv      # Salida de Cargar_datos.py
├── Observaciones_calidad_datos.csv  # Resumen de calidad de datos
├── Cargar_datos.py
├── Comprension_eda.py
├── ft_engineering.py
├── model_training_evaluation.py
├── model_deploy.py
├── API.py
├── enviar_predicciones_API.py
├── model_monitoring.py
├── APP.py
├── resultados_eda_entrega/          # Reportes, CSV y graficos/ (10 imagenes) del EDA
├── resultados_feature_engineering/  # X/y train-test, pipeline_preprocesamiento.joblib
├── resultados_model_training_evaluation/ # Modelos, matrices de confusion, curva ROC
├── resultados_model_deploy/         # Predicciones y reportes de deployment
├── resultados_model_monitoring/     # Metricas, drift, alertas y analisis temporal
│   └── data_drift_graficos/
├── modelo_final.joblib              # Modelo seleccionado (raiz y en resultados)
├── predicciones_api_monitoring.csv  # Historico de predicciones servidas por la API
├── Dockerfile
├── requirements.txt
├── .gitignore
└── README.md
```

Esta estructura no debe modificarse: los pipelines de validación de Jenkins dependen de ella para el paso a producción.

Diagrama de arquitectura



Flujo de artefactos y scripts del proyecto, extremo a extremo.

Datos de entrada y preparación — Cargar_datos.py

La base original Base_de_datos.csv contiene 10.763 registros y 23 columnas. La variable objetivo Pago_atiempo tiene 10.252 registros de clase 1 y 511 de clase 0, equivalentes aproximadamente a 95,25 % y 4,75 % respectivamente — un desbalance fuerte que se conserva durante el análisis y se aborda con métricas por clase y balanceadas en lugar de depender de Accuracy global.

Cargar_datos.py funciona como una barrera de control de calidad: lee la base, revisa dimensiones, tipos, nulos y duplicados, y aplica validaciones específicas sobre fechas, edades, variables financieras, puntajes, saldos, categorías y el objetivo. Cuando encuentra valores claramente inválidos los convierte en nulos, en lugar de inventar un reemplazo. No elimina masivamente registros: conserva la estructura y deja los problemas detectados listos para que etapas posteriores los traten.

Validaciones aplicadas

- Edades fuera del rango 18–100
- Valores negativos en variables donde no son válidos
- Salarios menores o iguales a cero
- Cuotas superiores al salario
- Puntajes negativos
- Saldos inconsistentes (p. ej. saldo principal o de mora mayor al saldo total)
- Categorías de tendencia_ingresos fuera de las tres esperadas (Creciente, Decreciente, Estable)
- Valores inválidos de Pago_atiempo
- Valores extremos mediante el criterio del rango intercuartílico (IQR)

El script también crea variables derivadas iniciales — año, mes, día de la semana y hora del préstamo; relación cuota/salario; relación deuda/salario; saldo pendiente estimado — y convierte los infinitos resultantes en nulos.

Resultados de la ejecución documentada

Indicador	Valor
Registros	10.763
Columnas	23
Valores nulos iniciales detectados	7.175
Filas completamente duplicadas	0
Edades fuera de rango (18–100)	150
Puntajes negativos	135
Puntajes Datacrédito negativos	1
Salarios ≤ 0	24
Cuotas superiores al salario	11
Valores de tendencia_ingresos fuera de categoría	58

Salidas: Base_de_datos_preparada.csv y Observaciones_calidad_datos.csv.

¿Por qué antes del EDA? Para evitar que el análisis exploratorio mezcle datos originales con errores ya identificables. Así el EDA recibe una versión consistente y puede concentrarse en describir distribuciones y patrones, en vez de repetir controles básicos de integridad.

Análisis exploratorio de datos (EDA) — `Comprension_eda.py`

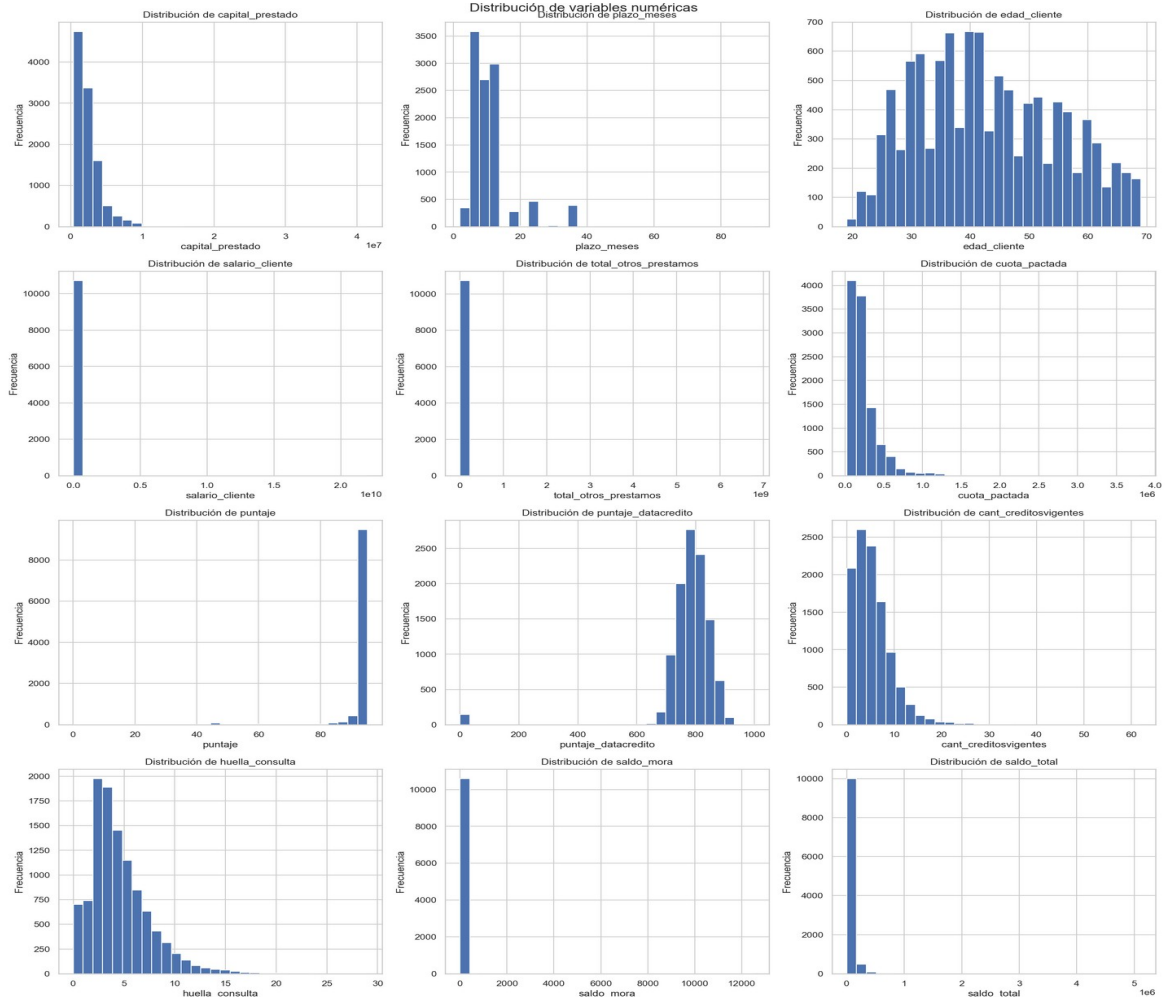
Toma `Base_de_datos_preparada.csv` y construye un análisis exploratorio reproducible. Al iniciar, elimina y vuelve a crear la carpeta `resultados_eda_entrega/` para que los resultados de una ejecución no se mezclen con archivos de una ejecución anterior.

El EDA no entrena el modelo: describe la estructura de los datos, caracteriza variables numéricas y categóricas, revisa el objetivo, estudia relaciones con `Pago_atiempo`, observa correlaciones, detecta comportamientos atípicos y revisa la dimensión temporal.

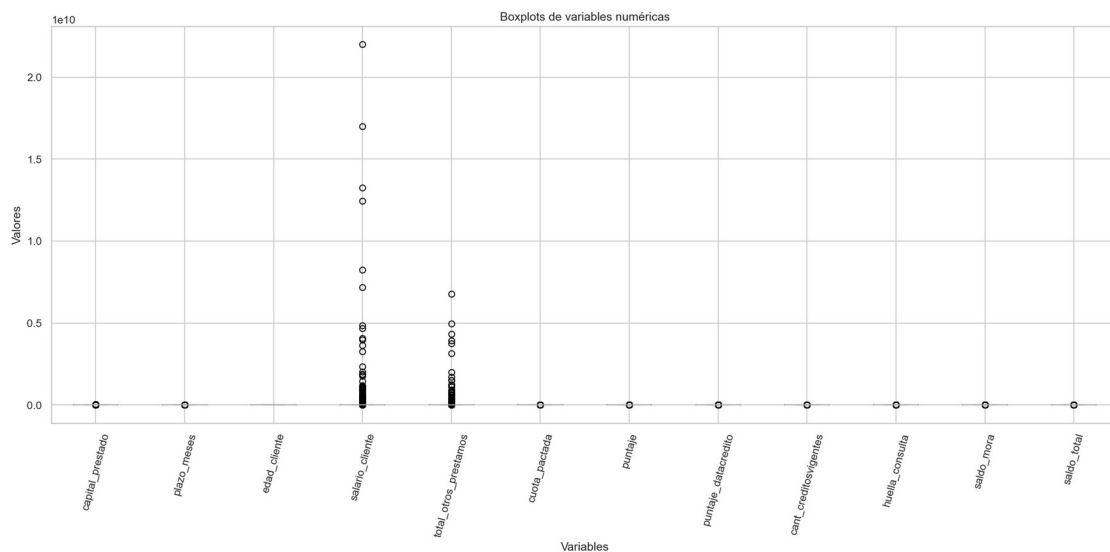
Qué analiza

- Resumen general (registros, columnas, celdas, nulos, clasificación de variables)
- Calidad de datos por columna y distribución del objetivo
- Estadísticas descriptivas y frecuencias de variables categóricas
- Comparación de variables numéricas y categóricas según `Pago_atiempo`
- Correlaciones entre variables numéricas y con el objetivo
- Valores extremos y comportamiento temporal de los préstamos

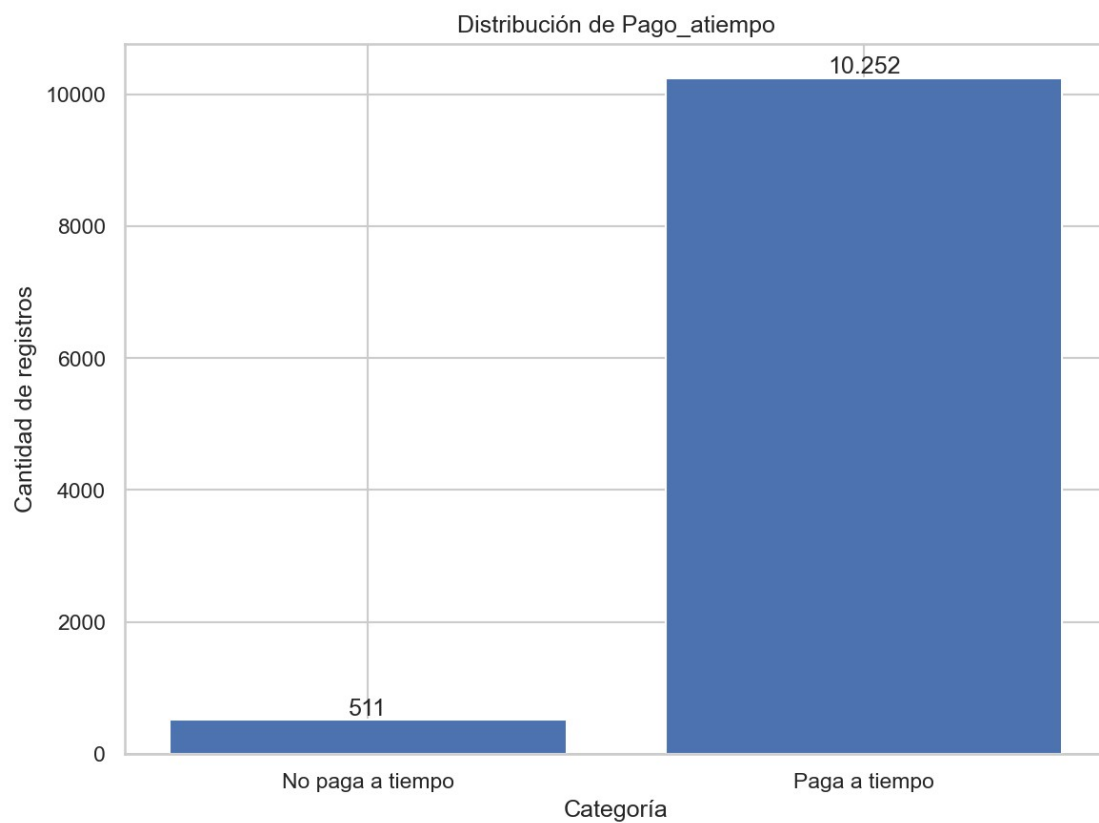
Los 10 gráficos generados (resultados_eda_entrega/graficos/)



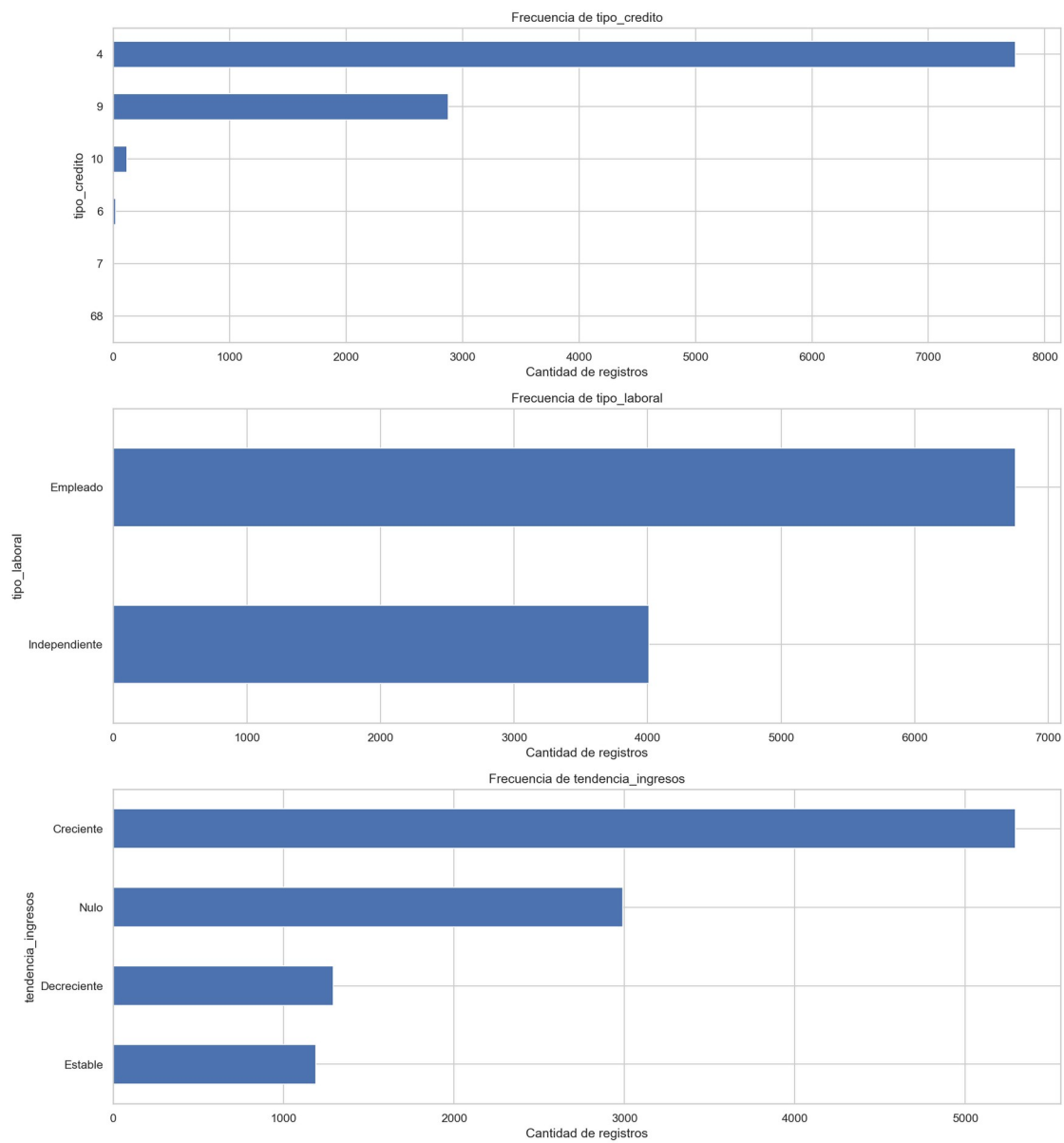
01_distribuciones_numericas.png — Histogramas de las variables numéricas principales (capital prestado, plazo, edad, salario, puntajes, saldos). Se observa fuerte concentración y asimetría en variables financieras, y outliers extremos en salario_cliente y total_otros_prestamos.



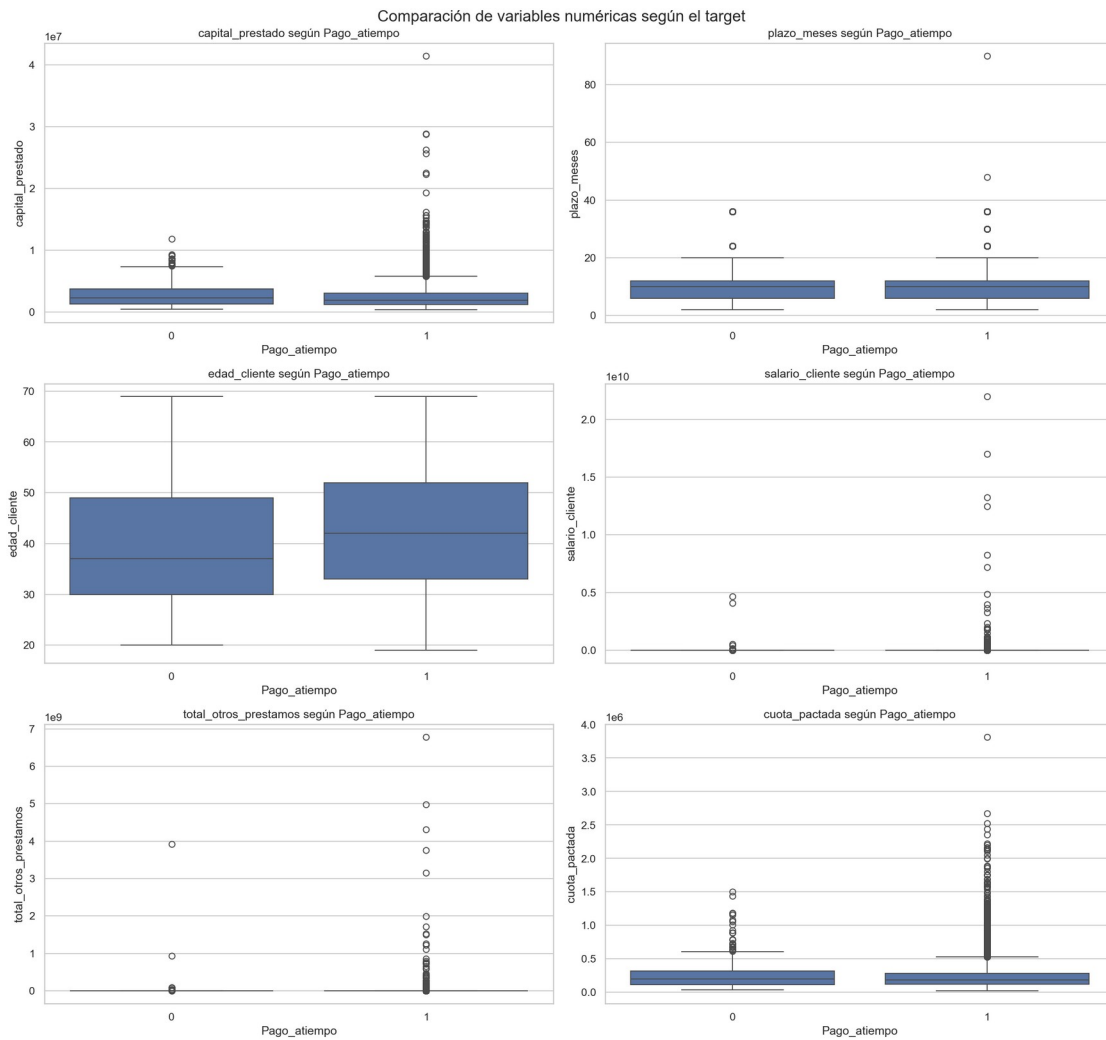
02_boxplots_numericos.png — Boxplots comparativos de las variables numéricas en una misma escala, que evidencian valores atípicos muy alejados en salario_cliente y total_otros_prestamos frente al resto de variables.



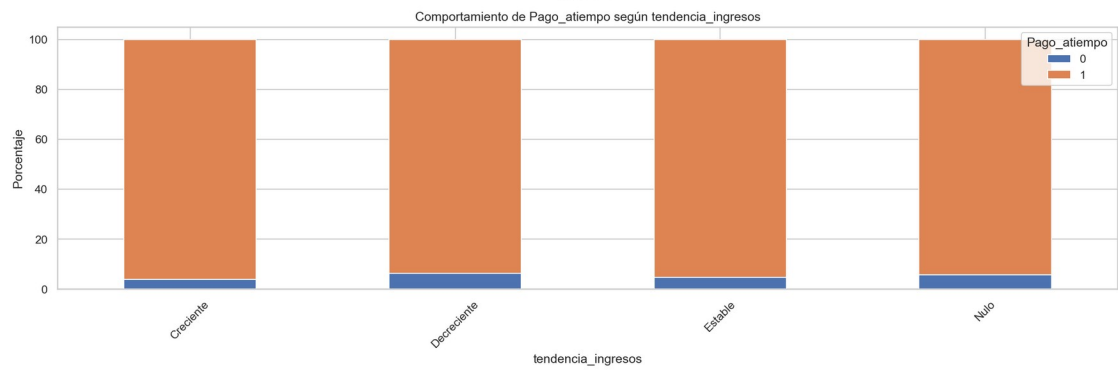
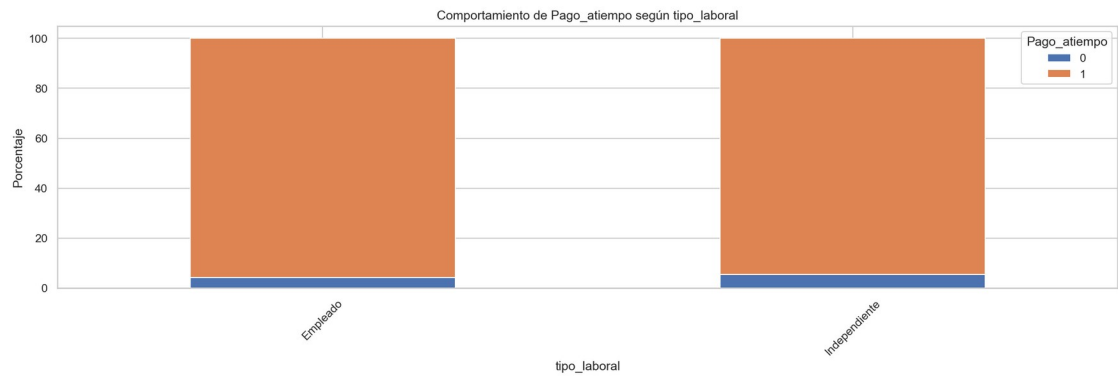
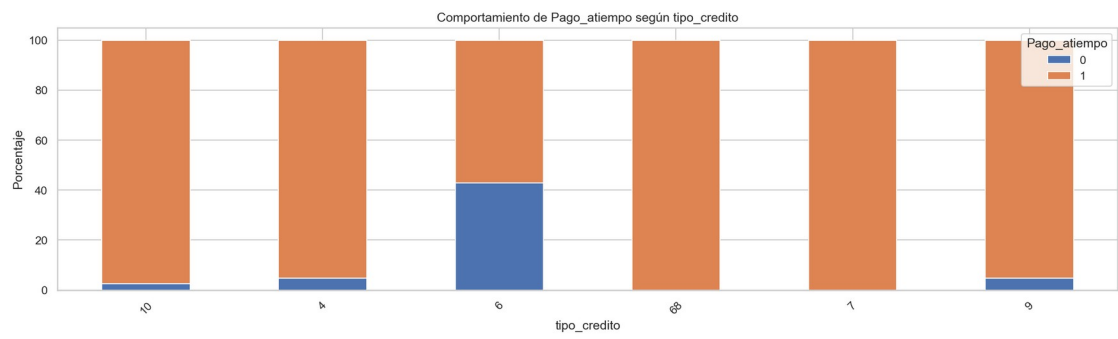
03_distribucion_target.png — Distribución de Pago_atiempo: 10.252 registros de pago a tiempo frente a 511 de no pago a tiempo. Es el gráfico más directo para mostrar el desbalance del objetivo (cantidades, no probabilidades de predicciones futuras).



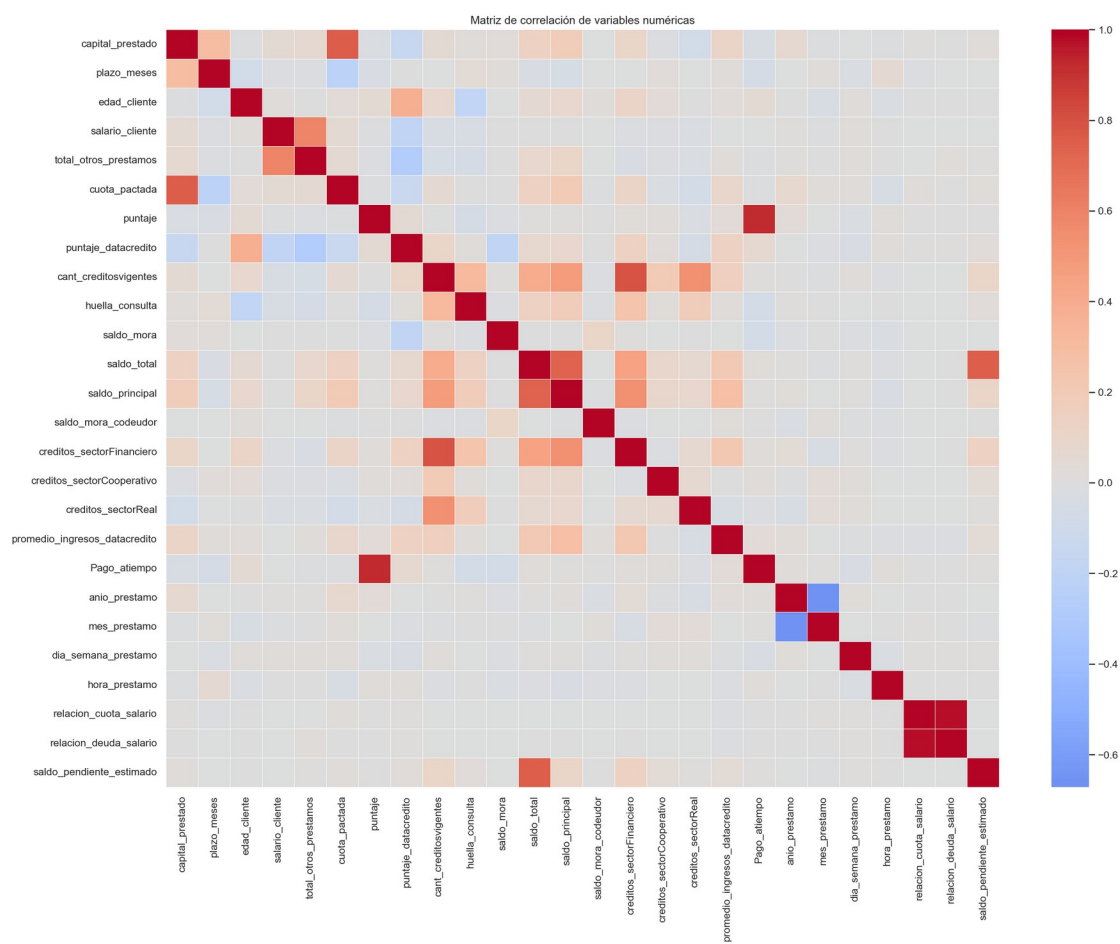
04_frecuencias_categoricas.png — Frecuencias de tipo_credito, tipo_laboral y tendencia_ingresos. tipo_credito está dominado por dos categorías; tipo_laboral se concentra en 'Empleado'; tendencia_ingresos se concentra en 'Creciente'.



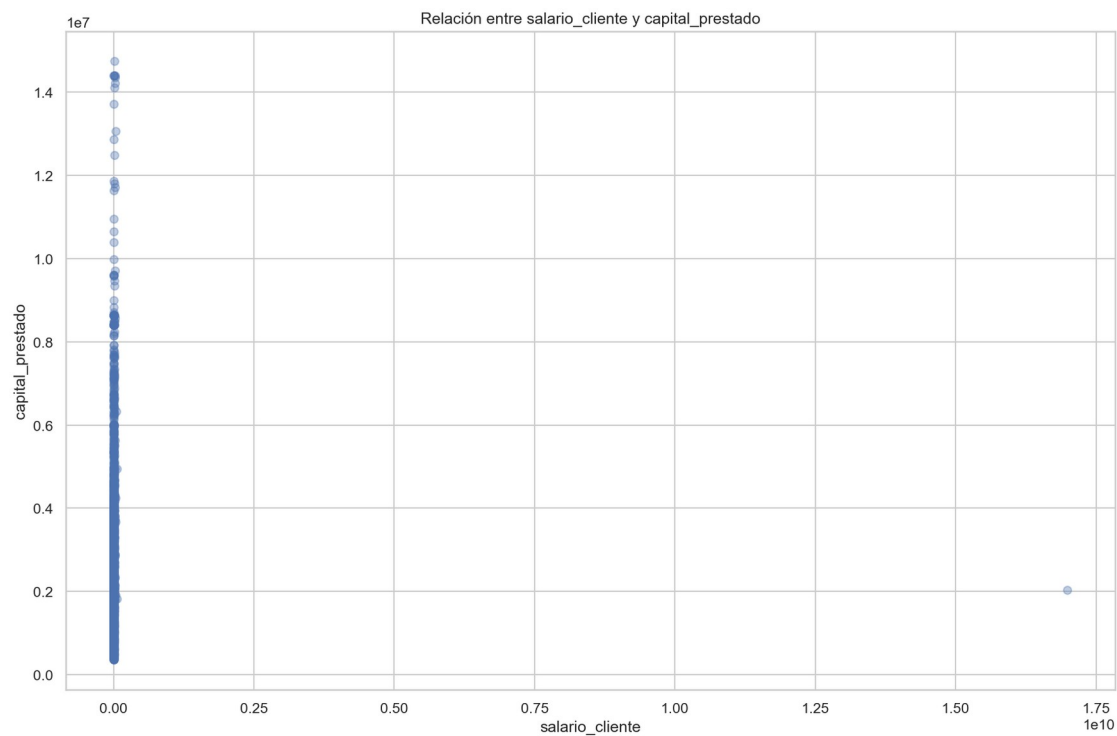
05_numericas_por_target.png — Boxplots de variables numéricas (*capital prestado, plazo, edad, salario, otros préstamos, cuota pactada*) separados por clase de *Pago_atiempo*, para observar diferencias de distribución entre pagadores e incumplidos.



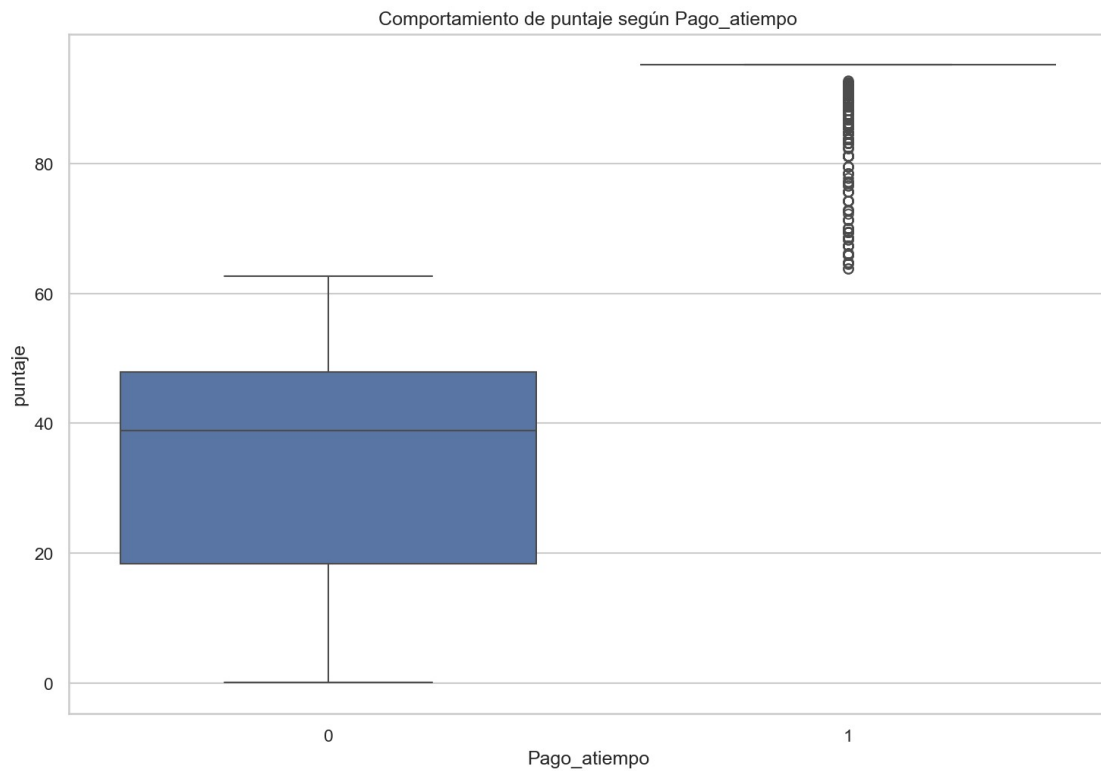
06_categoricas_por_target.png — Porcentaje de Pago_atiempo dentro de cada categoría de tipo_credito, tipo_laboral y tendencia_ingresos. El tipo de crédito código 6 muestra una proporción de incumplimiento notablemente mayor que el resto.



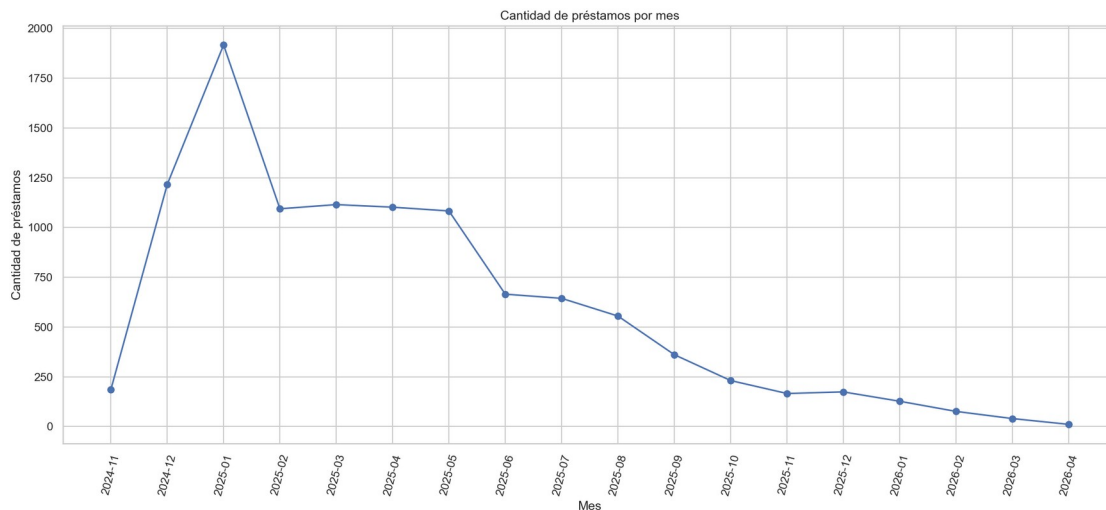
07_matriz_correlacion.png — Matriz de correlación de todas las variables numéricas, incluidas las derivadas. puntaje muestra la correlación más alta con Pago_atiempo, lo que refuerza la decisión de excluirla por posible fuga de información.



08_relaciones_financieras.png — Relación entre salario_cliente y capital_prestado. La nube de puntos muestra una relación poco definida y fuertemente afectada por valores extremos de salario.



09_comportamiento_puntajes.png — Comportamiento de puntaje según Pago_atiempo: los clientes que pagan a tiempo (clase 1) se concentran en puntajes altos, mientras que la clase 0 muestra una distribución mucho más amplia y baja.



10_comportamiento_temporal.png — Cantidad de préstamos por mes. Se observa un pico entre diciembre y enero, seguido de una tendencia decreciente sostenida en los meses siguientes.

Reportes y CSV generados

Reporte_EDA.xlsx, Reporte_EDA.txt, resumen_inicial.csv, resumen_outliers.csv, resumen_temporal.csv, calidad_datos.csv, clasificacion_variables.csv, distribucion_target.csv, estadisticas_numericas.csv, frecuencias_categoricas.csv, comparacion_numerica_target.csv, comportamiento_categoricas_target.csv, correlaciones.csv, correlaciones_target.csv, hallazgos.csv, matriz_correlacion.csv.

Hallazgos que pasan a la siguiente etapa

El objetivo está fuertemente desbalanceado; existen valores faltantes y comportamientos financieros que deben tratarse en el preprocesamiento; puntaje se identifica como variable potencialmente problemática por posible fuga de información y se excluye del modelado (ver secciones 6 y 13).

Correlación, diferencias entre grupos o alta importancia posterior de una variable no deben interpretarse automáticamente como causalidad. El EDA describe relaciones observadas y apoya decisiones técnicas; no demuestra que una variable provoque el incumplimiento.

Ingeniería de características — ft_engineering.py

Convierte la base preparada en una representación adecuada para entrenar modelos: el modelo no puede recibir directamente una mezcla de fechas, texto, valores faltantes y escalas financieras diferentes.

Creación de características

- fecha_prestamo se transforma en año, mes, día, día de la semana, semana del año, trimestre, hora y una variable binaria de fin de semana; la fecha original se elimina después.
- Variables financieras derivadas: relacion_cuota_salario, relacion_deuda_salario, saldo_pendiente_estimado.
- tipo_credito se convierte a variable categórica explícita.

Separación y prevención de fuga de información: Pago_atiempo se separa como objetivo; puntaje se excluye explícitamente por posible data leakage (ver sección 13).

División train/test: train_test_split con 80 % entrenamiento / 20 % prueba, random_state=42, stratify=y. Sobre 10.763 registros esto equivale a 8.610 registros de entrenamiento y 2.153 de prueba.

Preprocesamiento (ColumnTransformer)

- Numéricas → imputación por mediana + StandardScaler
- Categóricas → imputación por la categoría más frecuente + OneHotEncoder(handle_unknown="ignore")
- El preprocesador se ajusta únicamente con entrenamiento y luego se aplica a prueba, evitando fuga de información del conjunto de prueba.

Tras separar objetivo, puntaje y la fecha, quedan 27 variables predictoras (24 numéricas + 3 categóricas). Con la codificación One-Hot de las categorías presentes, la representación transformada llega a 35 características.

Salidas (resultados_feature_engineering/): X_train_transformado.csv, X_test_transformado.csv, y_train.csv, y_test.csv, pipeline_preprocesamiento.joblib, nombres_caracteristicas.csv, resumen_valores_nulos.csv, resumen_variables.csv, resumen_feature_engineering.txt.

Entrenamiento y evaluación de modelos — model_training_evaluation.py

Recibe los conjuntos ya transformados y entrena tres algoritmos:

Modelo	Configuración documentada en el código	Función en el proyecto
Regresión Logística	class_weight="balanced", max_iter=2000	Modelo de referencia (técnica lineal)
Random Forest	300 árboles, class_weight="balanced"	Enfoque no lineal basado en ensamble
SVM calibrado	SVC balanceado + CalibratedClassifierCV (sigmoid, cv=3)	Comparación con otra forma de separación de clases

class_weight="balanced" se usa en los modelos que lo soportan para considerar el desbalance durante el entrenamiento. La SVM base se envuelve en CalibratedClassifierCV para poder obtener probabilidades (predict_proba).

¿Por qué varios modelos? No se asume de antemano que una técnica será adecuada. Los tres se entrenan sobre los mismos datos y se evalúan con una estructura común, para poder seleccionar el modelo final según el objetivo del proyecto: detectar registros de la clase 0 (posible incumplimiento).

Métricas calculadas: Accuracy, Balanced Accuracy, ROC-AUC, PR-AUC, Precision/Recall/F1 por clase (0 y 1) y macro, matrices de confusión. La validación cruzada usa 5 particiones y F1-score macro como métrica, para observar el comportamiento en distintas divisiones del entrenamiento.

Criterio de selección del modelo final: el código ordena los modelos por (1) Recall de la clase 0, (2) F1-score de la clase 0 y (3) Balanced Accuracy. El primero según ese orden se guarda como modelo_final.joblib, tanto en resultados_model_training_evaluation/ como en la carpeta raíz del proyecto.

Este criterio refleja una decisión metodológica: interesa especialmente no perder la capacidad de identificar los registros 'No paga a tiempo'. Las métricas adicionales siguen siendo relevantes para entender el comportamiento global y el equilibrio entre clases.

Completar con los valores exactos de resultados_model_training_evaluation/resultados_modelos.csv al ejecutar el pipeline en el entorno de revisión.

Model Deployment — model_deploy.py

Reconstruye las mismas variables derivadas necesarias para el modelo, separa Pago_atiempo cuando está disponible, elimina puntaje y la fecha original, transforma los datos con el pipeline guardado y verifica que la cantidad de características resultante coincida con lo que espera el modelo.

Después genera la predicción, obtiene probabilidades por clase cuando el modelo lo permite, y crea una interpretación legible: clase 0 → 'Posible incumplimiento', clase 1 → 'Pago a tiempo'. Si existen valores reales de Pago_atiempo en los datos de entrada, también calcula Accuracy, Balanced Accuracy, Precision, Recall y F1 por clase, y una matriz de confusión.

API de predicción (FastAPI) — API.py

Convierte el modelo en un servicio HTTP. Al iniciar, carga modelo_final.joblib y resultados_feature_engineering/pipeline_preprocesamiento.joblib. Antes de predecir, reconstruye las mismas variables derivadas usadas en el feature engineering (temporales, relación cuota/salario, relación deuda/salario, saldo pendiente estimado), aplica el mismo pipeline y ejecuta el modelo — reproduciendo exactamente las transformaciones usadas en el entrenamiento.

Endpoints

Endpoint	Método	Función
/	GET	Confirma que la API está funcionando y lista los endpoints disponibles
/health	GET	Comprueba que la API, el modelo y el pipeline estén cargados
/predict	POST	Procesa un solo registro
/predict/batch	POST	Procesa una lista de registros

Respuesta de la API

Cada predicción devuelve: un identificador UUID (identifica la predicción; no es un mecanismo de cifrado), la clase predicha, el resultado legible, la probabilidad de incumplimiento, la probabilidad de pago a tiempo, el nivel de riesgo y el tiempo de predicción en milisegundos.

Umbral de nivel de riesgo (según probabilidad de incumplimiento)

Probabilidad de incumplimiento	Nivel de riesgo
< 0,40	Bajo
0,40 - < 0,70	Medio
≥ 0,70	Alto

Cada predicción se agrega a predicciones_api_monitoring.csv — este archivo conecta directamente la API con la etapa de monitoreo.

Ejemplo de solicitud (POST /predict/batch)

Úsalo para probar el servicio manualmente desde <http://127.0.0.1:8000/docs> (paso 7 de la ejecución rápida):

```
{
  "registros": [
    {
      "tipo_credito": "Libre inversión",
      "fecha_prestamo": "2024-03-15",
      "capital_prestado": 5000000,
      "plazo_meses": 24,
      "edad_cliente": 35,
      "tipo_laboral": "Empleado",
      "salario_cliente": 2500000,
      "total_otros_prestamos": 800000,
      "cuota_pactada": 250000,
      "puntaje_datacredito": 720,
      "cant_creditosvigentes": 2,
      "huella_consulta": 1,
      "saldo_mora": 0,
      "saldo_total": 800000,
      "saldo_principal": 700000,
      "saldo_mora_codeudor": 0,
      "creditos_sectorFinanciero": 1,
      "creditos_sectorCooperativo": 0,
      "creditos_sectorReal": 1,
      "promedio_ingresos_datacredito": 2400000,
      "tendencia_ingresos": "Estable"
    }
  ]
}
```



Documentación interactiva de la API (Swagger UI) — <http://127.0.0.1:8000/docs>

Envío de registros a la API — `enviar_predicciones_API.py`

Simula la llegada de nuevos registros al servicio. Lee `Base_de_datos.csv`, comprueba que existan las columnas requeridas, selecciona hasta 100 registros con `random_state=42`, convierte tipos y completa valores faltantes antes de construir la solicitud (numéricos → mediana de la muestra; categóricos → "Desconocido"; fecha → formato definido; columnas enteras → redondeadas y convertidas).

Envía los registros al endpoint `/predict/batch`, por lo que la API debe estar corriendo antes de ejecutar este script. La respuesta se guarda como `predicciones_api_monitoring.csv`.

```
Terminal 1: uvicorn API:app --host 127.0.0.1 --port 8000
Terminal 2: python enviar_predicciones_API.py
```

La separación en dos terminales es intencional: una mantiene el servicio disponible y la otra actúa como cliente que envía los registros.

Model Monitoring y Data Drift — `model_monitoring.py`

Compara una base de referencia (`Base_de_datos_preparada.csv`) con las predicciones recientes generadas por la API (`predicciones_api_monitoring.csv`).

Validación previa: solo se consideran válidas para el monitoreo las predicciones donde la clase predicha es 0 o 1, las probabilidades están entre 0 y 1, y la suma de probabilidad de incumplimiento + probabilidad de pago a tiempo está aproximadamente entre 0,99 y 1,01.

Qué se monitorea

- Distribución de las clases predichas
- Probabilidad de incumplimiento (promedio, mediana, mínimo, máximo)
- Niveles de riesgo y porcentaje de riesgo alto

- Tiempo de predicción
- Cambios en las distribuciones de las variables respecto a la referencia
- Comportamiento temporal de las predicciones

Data Drift

Tipo de variable	Métricas utilizadas
Numéricas	Kolmogorov-Smirnov (KS test), PSI, Jensen-Shannon
Catóricas	PSI, Jensen-Shannon, Chi-cuadrado

Reglas de detección (entre otros criterios): $p\text{-valor} < 0,05$, $PSI \geq 0,25$ y $Jensen\text{-Shannon} \geq 0,10$, según el tipo de variable y la métrica.

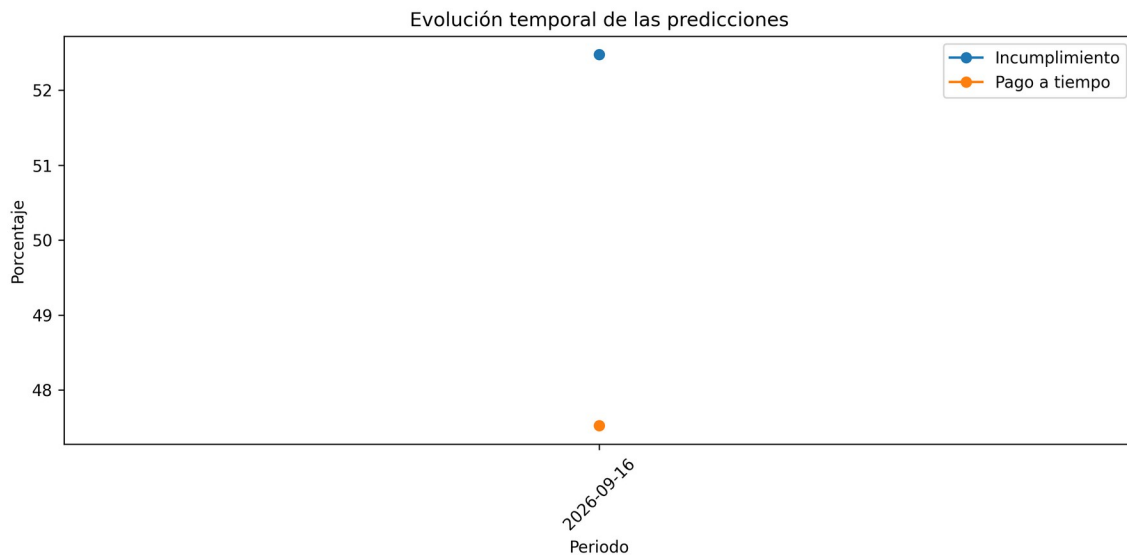
Resultado observado en la muestra actual (101 registros procesados por la API)

Indicador	Valor
Predicciones válidas analizadas	101
Posible incumplimiento	53 (52,48 %)
Pago a tiempo	48 (47,52 %)
Probabilidad de incumplimiento — media	51,92 %
Probabilidad de incumplimiento — mediana	51,12 %
Probabilidad de incumplimiento — mínimo	20,13 %
Probabilidad de incumplimiento — máximo	100 %
Riesgo bajo	24
Riesgo medio	65
Riesgo alto	12
Tiempo medio de predicción	27,21 ms
Variables evaluadas para drift	20
Variables con drift detectado	15 (75 %)

75 % significa "15 de 20 variables evaluadas" — no que el 75 % de los clientes haya cambiado de comportamiento, ni que el 75 % tenga riesgo alto. Este resultado corresponde a esta muestra y ejecución específicas; no debe interpretarse como una condición permanente del modelo o de datos futuros.

Entre las variables con señales de drift en esta ejecución destacan `tendencia_ingresos`, `promedio_ingresos_datacredito`, `cuota_pactada` y `tipo_credito` (detalle completo en `data_drift_resultados.csv`).

Estos porcentajes corresponden a las predicciones del modelo sobre la muestra procesada por la API. No deben confundirse con la distribución histórica del dataset original (95,25 % pagos a tiempo / 4,75 % no pagos a tiempo), ni implican que el 52,48 % de esos clientes efectivamente incumplirá.



analisi_temporal_predicciones.png — Evolución temporal de las predicciones (ejecución de ejemplo, muestra única).

Salidas (resultados_model_monitoring/): metricas_monitoring.csv, distribucion_predicciones.csv, predicciones_validas_monitoring.csv, resumen_niveles_riesgo.csv, alertas_monitoring.txt, data_drift_resultados.csv, data_drift_alertas.txt, variables_con_drift.csv, data_drift_graficos/, analisis_temporal.csv, analisis_temporal_predicciones.png.

Aplicación de Streamlit — APP.py

Construye una interfaz para consultar los resultados del monitoring. No vuelve a entrenar el modelo: lee los archivos generados por model_monitoring.py y los presenta visualmente.

Secciones

Sección	Contenido
Resumen	Total de predicciones, % de incumplimiento, probabilidad promedio, cantidad de riesgo alto, tiempo promedio, y tres gráficos: distribución de predicciones, distribución de probabilidad de incumplimiento y distribución de niveles de riesgo
Predicciones	Filtro por nivel de riesgo y por rango de probabilidad de incumplimiento
Data Drift	Variables monitorizadas, variables con drift, tabla completa de resultados y gráficos de data_drift_graficos/
Análisis temporal	Tabla temporal y gráfico analisis_temporal_predicciones.png

Streamlit funciona como capa de visualización del monitoring: no sustituye los scripts anteriores, consume sus resultados.

Monitoreo del Modelo de Crédito

Dashboard de seguimiento del modelo de predicción de incumplimiento de créditos.

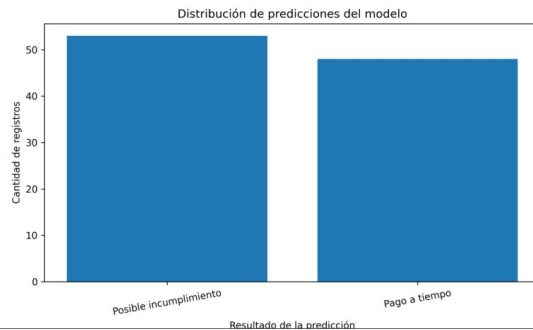
Permite consultar las predicciones generadas por la API, analizar el nivel de riesgo y revisar posibles cambios en la distribución de los datos mediante Data Drift.

Resumen del monitoreo

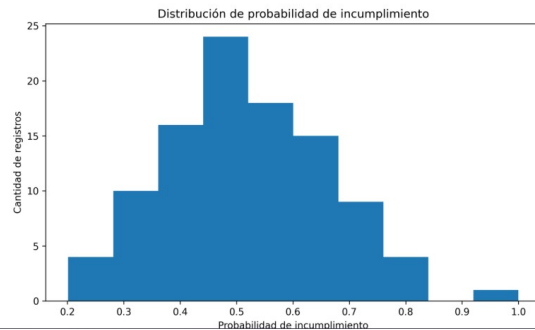
Total predicciones	% Incumplimiento	Probabilidad promedio	Riesgo alto	Tiempo promedio
101	52.48%	51.92%	12	27.21 ms

Distribuciones del modelo

Distribución de predicciones



Probabilidad de incumplimiento



Tablero de monitoreo — APP.py (Streamlit), sección Resumen.

Consideraciones metodológicas

- **El desbalance de clases:** es una característica central del problema. Por eso se usa `class_weight="balanced"` en Regresión Logística, Random Forest y en la SVM base, y además se evalúan las clases por separado.
- **La exclusión de puntaje:** se realiza por posible fuga de información. No significa que el puntaje sea irrelevante — se consideró metodológicamente más seguro no usarlo como predictor en esta implementación.
- **Las probabilidades:** producidas por el modelo son salidas del clasificador para la muestra evaluada. El nivel de riesgo es una categorización definida por los umbrales del servicio y no constituye, por sí misma, una conclusión causal sobre el comportamiento futuro de una persona.
- **El Data Drift:** tampoco demuestra por sí mismo que el modelo haya empeorado: indica que las distribuciones comparadas presentan diferencias bajo las métricas y umbrales utilizados. Evaluar el impacto real sobre el rendimiento requiere, posteriormente, contar con resultados reales observados para las predicciones actuales.

Contenerización con Docker

El proyecto incorpora Docker para empaquetar y ejecutar de forma reproducible el servicio de predicción (la API FastAPI, sus dependencias y los artefactos necesarios para inferencia).

Dockerfile

```
FROM python:3.13-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY API.py .
COPY modelo_final.joblib .
COPY resultados_feature_engineering/pipeline_preprocesamiento.joblib .

EXPOSE 8000

CMD ["uvicorn", "API:app", "--host", "0.0.0.0", "--port", "8000"]
```

Usa python:3.13-slim como imagen base, define /app como directorio de trabajo, copia requirements.txt e instala dependencias, e incorpora únicamente API.py, modelo_final.joblib y pipeline_preprocesamiento.joblib — lo mínimo necesario para servir el modelo. Expone el puerto 8000 y arranca Uvicorn automáticamente al iniciar el contenedor.

Construcción de la imagen

Desde la carpeta raíz del proyecto (donde están el Dockerfile, requirements.txt, el modelo y los demás archivos requeridos):

```
docker build -t modelo-creditos-api .
```

Ejecución del contenedor

```
docker run -d -p 8000:8000 --name modelo-creditos-api modelo-creditos-api
```

- -d ejecuta el contenedor en segundo plano.
- -p 8000:8000 conecta el puerto 8000 del host con el puerto 8000 de FastAPI dentro del contenedor.
- modelo-creditos-api (como --name) permite identificar el contenedor con los comandos de Docker.

Verificación del servicio

- Documentación interactiva: <http://127.0.0.1:8000/docs>
- Estado del servicio: <http://127.0.0.1:8000/health> — confirma que el servicio, el modelo y el pipeline de preprocesamiento se cargaron correctamente.

Versionamiento con Git y GitHub

El proyecto usa Git para controlar versiones y mantener el historial de cambios. El repositorio se inicializa desde la carpeta raíz, conservando la estructura de archivos necesaria para que los scripts se ejecuten correctamente.

Flujo de ramas: main es la rama principal y representa la versión integrada del proyecto. Los cambios se desarrollan en ramas de trabajo independientes y se integran a main mediante merge una vez completados y revisados, evitando modificar directamente la versión principal durante cada etapa de desarrollo.

Historial de entregas (commits/ramas realizados)

Entrega	Contenido incorporado
1	Cargar_datos.py, Comprension_eda.py, ft_engineering.py y sus resultados (Base_de_datos_preparada.csv, Observaciones_calidad_datos.csv, resultados_eda_entrega/, resultados_feature_engineering/)
2	model_training_evaluation.py, model_deploy.py y sus resultados (resultados_model_training_evaluation/, resultados_model_deploy/, modelo_final.joblib)
3	API.py, APP.py, Dockerfile, enviar_predicciones_API.py, model_monitoring.py y todos sus resultados correspondientes (predicciones_api_monitoring.csv, resultados_model_monitoring/)

Cada commit permite identificar los cambios realizados en su etapa, facilitando el seguimiento de la evolución del código y la recuperación de versiones anteriores cuando sea necesario.

Estructura y control de archivos — .gitignore

```
__pycache__/  
*.pyc  
  
.venv/  
venv/  
  
.env  
*.log  
  
.streamlit/secrets.toml
```

Los resultados y artefactos que forman parte de la entrega sí se conservan en el repositorio: las carpetas resultados_*, modelo_final.joblib y predicciones_api_monitoring.csv.

Repositorio remoto

El proyecto se sincroniza con un repositorio remoto en GitHub, lo que permite conservar código y archivos versionados en un repositorio central, facilitar la entrega y mantener un historial de modificaciones. La estructura interna debe conservarse también dentro del repositorio remoto: no deben cambiarse nombres o ubicaciones de archivos utilizados por los scripts sin actualizar las rutas correspondientes en el código.

Stack tecnológico

- **Lenguaje:** Python 3.13
- **Análisis y manipulación de datos:** pandas, numpy, tabulate
- **Visualización:** matplotlib, seaborn
- **Modelamiento:** scikit-learn, xgboost, joblib
- **Servicio del modelo:** FastAPI, Pydantic, Uvicorn
- **Monitoreo:** SciPy (pruebas estadísticas), Streamlit
- **Contenerización:** Docker

- **Control de versiones:** Git / GitHub

Conclusión

El proyecto construye un flujo completo y trazable de Machine Learning para la predicción de incumplimiento de créditos. Comienza con una revisión de calidad que transforma la base original en un dataset preparado; continúa con un EDA que documenta la estructura, distribución y relaciones de los datos; y convierte esa información en características procesables mediante un pipeline reproducible.

La etapa de modelado compara tres enfoques —Regresión Logística, Random Forest y SVM calibrado— utilizando múltiples métricas y validación cruzada. La selección del modelo final prioriza la detección de la clase 'No paga a tiempo', y el modelo seleccionado se persiste para reutilizarlo sin repetir el entrenamiento.

A partir de ese artefacto, el proyecto pasa de un experimento de Machine Learning a un proceso de inferencia: deployment aplica el pipeline y el modelo a nuevos registros, FastAPI expone la predicción mediante endpoints (contenerizados con Docker), el script de envío simula el ingreso de nuevos datos, y monitoring analiza las predicciones y posibles cambios en las distribuciones. Finalmente, Streamlit reúne los resultados del monitoreo en una interfaz visual.

La solución no se limita al entrenamiento de un modelo: integra preparación, análisis, modelado, reutilización, servicio, monitoreo y visualización, manteniendo una relación explícita entre los archivos que producen y consumen cada etapa.
